Cowrie Honeypot Deployment and Troubleshooting in Proxmox

Project Overview

The goal of this project was to deploy a Cowrie SSH honeypot inside a Proxmox homelab environment. The honeypot was first deployed in an isolated internal network to safely understand how Cowrie works before considering exposing it to the internet.

The project demonstrates:

- Proxmox virtualization
- Ubuntu Server administration
- SSH configuration
- Honeypot deployment
- Security monitoring
- Troubleshooting Linux applications
- Attack simulation using Kali Linux

Environment

Hardware

Host Machine:

- HP EliteDesk 800 G6 Mini
- Intel i5-10600T (6 Cores / 12 Threads)
- 16 GB RAM
- 512 GB NVMe SSD

Hypervisor

- Proxmox VE

Virtual Machines

Kali Linux

Purpose:

- Attack machine
- Penetration testing
- Honeypot interaction

Ubuntu Server 24.04

Purpose:

- Cowrie Honeypot Host

Initial Problems Encountered

Problem 1: Ubuntu 26.04 Installation

Initially Cowrie was installed on Ubuntu Server 26.04.

Several issues occurred:

- Tutorials no longer matched the newest Cowrie repository structure.
- Traditional Cowrie commands failed.
- Configuration files were not located where expected.
- Startup scripts used in older guides no longer existed.

Example:

Expected:

bin/cowrie start

Actual:

bin/cowrie: No such file or directory

Resolution

A fresh Ubuntu Server 24.04 LTS VM was created because:

- Most Cowrie documentation targets Ubuntu 24.04.
- Community support is stronger.
- Package compatibility is more predictable.

Creating the Ubuntu 24.04 VM

VM Configuration

Name:
web01

Operating System:
Ubuntu Server 24.04 LTS

System Settings:

Machine:
q35

BIOS:
OVMF (UEFI)

SCSI Controller:
VirtIO SCSI single

Disk Settings:

Bus:
SCSI

Storage:
local-lvm

Disk Size:
20 GB

CPU:

Sockets:
1

Cores:
2

Type:
host

Memory:

2048 MB

Network:

Bridge:
vmbr0

Model:
VirtIO

Firewall:
Enabled

Ubuntu Installation

During installation:

Language:
English

Keyboard:
US

Proxy:
Blank

Ubuntu Mirror:
Default

Storage:
Use Entire Disk

OpenSSH:
Installed

Hostname:
web01

Username:
alex

Post Installation Configuration

Update system:

sudo apt update

sudo apt upgrade -y

Install dependencies:

sudo apt install git python3 python3-pip python3-venv libssl-dev libffi-dev build-essential -y

Verify network connectivity:

ip a

ping -c 4 google.com

Expected Result:

- Valid IP address
- Successful ping replies

Installing Cowrie

Create Dedicated User

sudo adduser --disabled-password cowrie

Switch user:

sudo su - cowrie

Expected Prompt:

cowrie@web01:~$

Clone Repository

git clone https://github.com/cowrie/cowrie.git

Move into project directory:

cd cowrie

Verify:

pwd

Expected:

/home/cowrie/cowrie

Create Python Virtual Environment

python3 -m venv cowrie-env

Activate environment:

source cowrie-env/bin/activate

Expected:

(cowrie-env)

Install Python Dependencies

python -m pip install --upgrade pip

pip install -r requirements.txt

Problem 2: Configuration File Missing

Attempted Command:

cp etc/cowrie.cfg.dist etc/cowrie.cfg

Error:

No such file or directory

Cause

The latest Cowrie repository structure had changed.

Configuration files were no longer stored in the traditional location.

Investigation

Search for configuration files:

find . -name "*.cfg*"

Result:

./src/cowrie/data/etc/cowrie.cfg.dist

Resolution

Create local configuration directory:

mkdir -p etc

Copy configuration file:

cp src/cowrie/data/etc/cowrie.cfg.dist etc/cowrie.cfg

Verify:

ls etc

Expected:

cowrie.cfg

Problem 3: Startup Script Missing

Attempted:

bin/cowrie start

Error:

No such file or directory

Cause

Newer Cowrie versions use a different project structure.

Older startup instructions no longer applied.

Resolution

Install Cowrie into the virtual environment:

pip install -e .

This allowed Cowrie components to be properly registered within Python.

Verifying Cowrie

Start monitoring logs:

tail -f var/log/cowrie/cowrie.log

Expected:

Live event logging window.

This confirmed Cowrie was operational.

Attack Simulation

Attacker Machine

Kali Linux VM

SSH Attack

Initial test:

ssh root@

Result:

No Cowrie activity.

Cause

Connection was targeting Ubuntu's real SSH service.

Cowrie was listening on:

Port 2222

Correct test:

ssh root@ -p 2222

Successful Honeypot Interaction

Attempted passwords:

root

Result:

Failed

Attempted password:

password

Result:

Success

Cowrie granted access to a fake shell.

This behavior is intentional.

Cowrie:

- Simulates successful compromise
- Records activity
- Prevents access to the real system

Evidence of Success

Observed in Cowrie logs:

- New connection
- Username attempts
- Password attempts
- Successful login
- Session creation

Observed from Kali:

- Fake Linux shell
- Simulated filesystem
- Simulated commands

Result:

The honeypot successfully captured attacker activity while protecting the host system.

Skills Demonstrated

Virtualization:

- Proxmox VE
- VM creation
- Resource allocation

Linux Administration:

- Ubuntu Server
- User management
- Package installation

Networking:

- IP addressing
- SSH
- Virtual networking

Cybersecurity:

- Honeypot deployment
- SSH monitoring
- Attack simulation
- Log analysis

Troubleshooting:

- Repository structure changes
- Missing configuration files
- Missing startup scripts
- Service verification

Outcome

The Cowrie honeypot was successfully deployed in an isolated Proxmox lab environment. Attack traffic generated from Kali Linux was captured and logged, demonstrating the ability to monitor unauthorized access attempts without exposing a real system.

Future enhancements include:

- Wazuh SIEM integration
- pfSense firewall deployment
- Network segmentation
- Internet-facing honeypot deployment
- Centralized log collection
- Threat intelligence analysis